Chapter 2 – Linear Algebra for Machine Learning

Applications with Real Data in Python

Dr. Youssef SALMAN

Lebanese University – Faculty of Sciences (Section V)

M1: Applied Mathematics & Statistics for Computational Sciences

Academic Year 2026–2027

Learning Objectives

- Represent data using vectors, matrices and tensors.
- Perform matrix arithmetic: addition, multiplication, transpose, determinant and inverse.
- Interpret matrices as linear transformations and study kernel, image, rank and invertibility.
- Understand eigenvalues, eigenvectors, diagonalization and their applications.
- Use SVD and PCA for stable computation, compression and dimensionality reduction.
- Implement the main ideas in Python using NumPy and Scikit-Learn.

Outline I

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Why Linear Algebra for Machine Learning?

- Machine learning learns patterns from data and makes predictions or decisions.
- Linear algebra provides the language for high-dimensional data.
- Regression, deep learning, PCA, recommender systems and computer vision all depend on vectors and matrices.

Key idea: linear algebra is the computational backbone of machine learning.

Examples in Real Applications

- **Search engines:** matrix decompositions index documents.
- **Image compression:** linear transformations reduce size while preserving structure.
- **Recommendation systems:** vector similarity finds close users or products.
- **PCA:** eigenvalues/eigenvectors reduce dimensionality.
- **Deep learning:** each layer uses matrix multiplication.

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Vectors

Definition

A **vector** is an ordered collection of numbers representing one object with several features:

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^d.$$

Machine-learning example

A house described by area, number of bedrooms, age and distance to the center can be represented by

$$\boldsymbol{x} = \begin{bmatrix} 120 \\ 3 \\ 15 \\ 8 \end{bmatrix}.$$

Vector Operations

For $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$:

- **Norm:** $\|\boldsymbol{x}\| = \sqrt{x_1^2 + \cdots + x_d^2}$.
- **Dot product:** $\boldsymbol{x} \cdot \boldsymbol{y} = \sum_{i=1}^{d} x_i y_i$.
- **Cosine similarity:** $\cos\theta = \dfrac{\boldsymbol{x} \cdot \boldsymbol{y}}{\|\boldsymbol{x}\|\|\boldsymbol{y}\|}$.

Cosine similarity is used in information retrieval, recommendation systems and NLP.

Definition

A **matrix** is a rectangular array of numbers:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

Matrices represent collections of vectors or linear transformations.

Dataset Matrix

Suppose information is collected from four students:

$$X = \begin{bmatrix} 170 & 65 & 22 \\ 180 & 78 & 24 \\ 165 & 60 & 20 \\ 175 & 70 & 23 \end{bmatrix}.$$

- Each row is one student.
- Each column is one feature: height, weight, age.

In machine learning, datasets are almost always stored as matrices.

Matrix-Vector Product

If $A \in \mathbb{R}^{m \times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$, then

$$A\boldsymbol{x} \in \mathbb{R}^m.$$

It computes weighted combinations of the components of $\boldsymbol{x}$.

Neural network layer

$$\boldsymbol{z} = W\boldsymbol{x} + \boldsymbol{b}.$$

Here W is the weight matrix, $\boldsymbol{b}$ is the bias and $\boldsymbol{z}$ is the output before activation.

Definition

A **tensor** is a multidimensional array that generalizes scalars, vectors and matrices.

Object	Order	Example
Scalar	0	$x \in \mathbb{R}$
Vector	1	$\boldsymbol{x} \in \mathbb{R}^d$
Matrix	2	$A \in \mathbb{R}^{m \times n}$
Higher tensor	3+	$T \in \mathbb{R}^{3 \times 4 \times 2}$

TensorFlow and PyTorch manipulate data mainly as tensors.

Images and Videos as Tensors

- A color image 256×256 with RGB channels has shape $256 \times 256 \times 3$.
- A video with 100 frames of this size has shape $100 \times 256 \times 256 \times 3$.
- The dimensions correspond to frame number, height, width and color channel.

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Addition and Scalar Multiplication

Let $A = (a_{ij})$, $B = (b_{ij}) \in \mathbb{R}^{m \times n}$.

$$A + B = (a_{ij} + b_{ij}), \qquad \lambda A = (\lambda a_{ij}).$$

Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 1 \\ -2 & 3 \end{bmatrix}$$

$$A + B = \begin{bmatrix} 6 & 3 \\ 1 & 7 \end{bmatrix}, \qquad 2A = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}.$$

ML Meaning of Addition

Matrix addition combines datasets, updates parameters or adds intermediate outputs.

$$W_{new} = W_{old} - \eta \nabla L(W_{old}).$$

- W_{old}: current parameters.
- $\nabla L(W_{old})$: gradient of the loss.
- η: learning rate.

Matrix Multiplication

Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. Then $C = AB \in \mathbb{R}^{m \times p}$ has entries

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

The entry c_{ij} is the dot product of the i-th row of A and the j-th column of B.

$$\underbrace{A}_{m \times n} \underbrace{B}_{n \times p} = \underbrace{AB}_{m \times p}.$$

Matrix Multiplication Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 2 & 0 \\ 1 & 5 \end{bmatrix}.$$

$$AB = \begin{bmatrix} 1(2)+2(1) & 1(0)+2(5) \\ 3(2)+4(1) & 3(0)+4(5) \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 10 & 20 \end{bmatrix}.$$

- Associative: $(AB)C = A(BC)$.
- Generally not commutative: $AB \neq BA$.

Multiplication as Composition

If B acts first and A acts second, then

$$\boldsymbol{x} \mapsto B\boldsymbol{x} \mapsto A(B\boldsymbol{x}) = (AB)\boldsymbol{x}.$$

Therefore, matrix multiplication encodes the composition of transformations.

The rightmost matrix acts first.

Transpose

For $A = (a_{ij}) \in \mathbb{R}^{m \times n}$, the transpose $A^T \in \mathbb{R}^{n \times m}$ is defined by

$$(A^T)_{ij} = a_{ji}.$$

Example

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \Rightarrow \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Transpose Properties and ML Meaning

- $(A^T)^T = A$, $(A+B)^T = A^T + B^T$, $(\lambda A)^T = \lambda A^T$.
- $(AB)^T = B^T A^T$: order is reversed.
- A square matrix is symmetric if $A^T = A$.

For a data matrix $X \in \mathbb{R}^{m \times n}$:

$$X^T X \in \mathbb{R}^{n \times n} \text{ measures feature relationships,}$$

$$XX^T \in \mathbb{R}^{m \times m} \text{ measures observation relationships.}$$

Determinant

The determinant is defined only for square matrices.

Definition

For $A \in M_n(\mathbb{K})$, $\det(A)$ is the unique function that is linear in each column, changes sign when two columns are exchanged and satisfies $\det(I_n) = 1$.

For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$:

$$\det(A) = ad - bc.$$

Geometric Meaning of the Determinant

If the columns of A are vectors $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^2$, then

$$|\det(A)| = \text{area of the parallelogram generated by } \boldsymbol{u}, \boldsymbol{v}.$$

In $\mathbb{R}^3$, $|\det(A)|$ is the volume of the parallelepiped generated by the columns.

- $\det(A) > 0$: orientation preserved.
- $\det(A) < 0$: orientation reversed.
- $\det(A) = 0$: space collapses to lower dimension.

Determinant of a 3×3 Matrix

Let $A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$. Expanding along the first row:

$$\det(A) = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix}.$$

Thus,

$$\det(A) = a(ei - fh) - b(di - fg) + c(dh - eg).$$

Minors, Cofactors and Properties

- The minor M_{ij} is obtained by deleting row i and column j.
- The cofactor is $C_{ij} = (-1)^{i+j} M_{ij}$.
- Expansion: $\det(A) = \sum_{j=1}^{n} a_{ij} C_{ij}$.

Main properties:

$$\det(AB) = \det(A)\det(B), \quad \det(A^T) = \det(A), \quad \det(\lambda A) = \lambda^n \det(A).$$

$$A \text{ invertible} \iff \det(A) \neq 0.$$

- Exchanging two rows changes the sign.
- Multiplying one row by λ multiplies the determinant by λ.
- Adding a multiple of one row to another does not change the determinant.

If A is triangular, then

$$\det(A) = \prod_{i=1}^{n} a_{ii}.$$

Example

$$\begin{vmatrix} 2 & 1 & 4 \\ 0 & 3 & 5 \\ 0 & 0 & -2 \end{vmatrix} = 2(3)(-2) = -12.$$

Determinant in Machine Learning

- If columns of X are linearly dependent, then $\det(X^T X) = 0$.
- If $|\det(X^T X)|$ is close to zero, there may be strong multicollinearity.
- In a multivariate Gaussian, $\det(\Sigma)$ measures generalized variance.
- Change of variables: if $\boldsymbol{y} = A\boldsymbol{x}$, volume elements are multiplied by $|\det(A)|$.

Inverse of a Matrix

Definition

The inverse of $A \in M_n(\mathbb{K})$, denoted A^{-1}, satisfies

$$AA^{-1} = A^{-1}A = I_n.$$

Equivalent statements:

$$A \text{ invertible} \iff \det(A) \neq 0 \iff \operatorname{rank}(A) = n \iff \text{columns of } A \text{ are independent.}$$

Inverse of a 2×2 Matrix

For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, if $ad - bc \neq 0$, then

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Example

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

General Inverse Formula

The adjugate formula is

$$A^{-1} = \frac{1}{\det(A)} \operatorname{adj}(A), \qquad \det(A) \neq 0.$$

Procedure:

1. compute $\det(A)$ and verify it is nonzero;
2. compute all minors M_{ij};
3. compute cofactors $C_{ij} = (-1)^{i+j} M_{ij}$;
4. form the cofactor matrix C;
5. transpose it: $\operatorname{adj}(A) = C^T$.

Singular Matrix and Information Loss

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}, \qquad \det(B) = 1(4) - 2(2) = 0.$$

Therefore B is singular and has no inverse.

Geometrically, the transformation maps the plane onto a line. Information is lost and cannot be recovered.

- $(AB)^{-1} = B^{-1}A^{-1}$.
- $(A^T)^{-1} = (A^{-1})^T$.
- $(A^{-1})^{-1} = A$.
- $\det(A^{-1}) = 1/\det(A)$.

For $A\boldsymbol{x} = \boldsymbol{b}$, if A is invertible, then

$$\boldsymbol{x} = A^{-1}\boldsymbol{b}.$$

Numerical warning

In practice, solve linear systems without explicitly computing the inverse.

Application: Linear Regression

Ordinary least squares is formally written as

$$\widehat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \boldsymbol{y}.$$

If $X^T X$ is singular or nearly singular, coefficients are unstable.

Ridge regression stabilizes the problem:

$$\widehat{\boldsymbol{\beta}}_{ridge} = (X^T X + \lambda I)^{-1} X^T \boldsymbol{y}, \qquad \lambda > 0.$$

Numerical Computation

Instead of explicit inverses, use:

- **QR decomposition:** $X = QR$, then solve $R\widehat{\beta} = Q^T \boldsymbol{y}$.
- **SVD:** $X = U\Sigma V^T$, use pseudoinverse $X^+ = V\Sigma^+ U^T$.

QR is efficient for full-rank well-conditioned problems; SVD is more robust for ill-conditioned or rank-deficient problems.

Orthogonal Matrix

Definition

A square matrix $Q \in \mathbb{R}^{n \times n}$ is **orthogonal** if

$$Q^T Q = Q Q^T = I_n.$$

Equivalently, $Q^{-1} = Q^T$.

Its columns and rows are mutually orthogonal unit vectors. Orthogonal matrices preserve lengths and angles:

$$\|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|.$$

Example: Rotation Matrix

$$Q = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Then

$$Q^T Q = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = I_2.$$

This is a rotation of 90° counterclockwise.

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Definition

A mapping $T : \mathbb{R}^n \to \mathbb{R}^m$ is linear if

$$T(\boldsymbol{x} + \boldsymbol{y}) = T(\boldsymbol{x}) + T(\boldsymbol{y}), \qquad T(\alpha \boldsymbol{x}) = \alpha T(\boldsymbol{x}).$$

Equivalently,

$$T(\alpha \boldsymbol{x} + \beta \boldsymbol{y}) = \alpha T(\boldsymbol{x}) + \beta T(\boldsymbol{y}).$$

Matrix Representation

Every linear transformation can be represented by a matrix:

$$T(\boldsymbol{x}) = A\boldsymbol{x}.$$

Conversely, every matrix defines a linear transformation.

This equivalence allows us to study geometric transformations using matrix properties.

Geometric Interpretations in $\mathbb{R}^2$

Effect	Matrix / Formula
Scaling	$T(\boldsymbol{x}) = \alpha \boldsymbol{x}$
Reflection	$T(x, y) = (x, -y)$
Rotation	$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$
Shear	$S = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$
Projection	$P = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$

Kernel and Image

For $T : \mathbb{R}^n \to \mathbb{R}^m$:

$$\ker(T) = \{\boldsymbol{x} \in \mathbb{R}^n : T(\boldsymbol{x}) = \mathbf{0}\},$$
$$\mathrm{Im}(T) = \{T(\boldsymbol{x}) : \boldsymbol{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m.$$

- If $\ker(T) = \{0\}$, then T is injective.
- If $\mathrm{Im}(T) = \mathbb{R}^m$, then T is surjective.

Kernel = information lost. Image = possible outputs.

Definition

The rank of T is $\dim(\mathrm{Im}(T))$. It measures the number of independent directions preserved.

Theorem (Rank-Nullity)

For every linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$,

$$\dim(\ker T) + \dim(\mathrm{Im}\ T) = n.$$

If $T : \mathbb{R}^n \to \mathbb{R}^n$ and $\mathrm{rank}(T) = n$, then T is invertible.

Composition of Transformations

If $T_1(\boldsymbol{x}) = A\boldsymbol{x}$ and $T_2(\boldsymbol{x}) = B\boldsymbol{x}$, then

$$T_2(T_1(\boldsymbol{x})) = B(A\boldsymbol{x}) = (BA)\boldsymbol{x}.$$

Projection

$$P = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad P^2 = P, \qquad P^T = P.$$

Repeated projection does nothing new: P is idempotent.

Linear Transformations in ML

- Neural networks: each layer applies $W\boldsymbol{x} + \boldsymbol{b}$ followed by a nonlinearity.
- PCA: data are projected onto a lower-dimensional subspace.
- Computer vision: rotations, scalings and reflections act on pixel coordinates.
- Feature engineering: normalization and standardization are affine transformations.

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Definition

Let $A \in \mathbb{R}^{n \times n}$. A nonzero vector $\boldsymbol{v} \in \mathbb{R}^n$ or $\mathbb{C}^n$ is an eigenvector of A if there exists a scalar λ such that

$$A\boldsymbol{v} = \lambda \boldsymbol{v}.$$

The scalar λ is the eigenvalue.

Eigenvectors are directions preserved by A; eigenvalues are stretching/shrinking factors.

Existence

- Not every real matrix has real eigenvalues.
- Every square matrix has complex eigenvalues because the characteristic polynomial has roots in $\mathbb{C}$.
- Real symmetric matrices have real eigenvalues and an orthonormal eigenbasis.
- Orthogonal matrices have eigenvalues with modulus 1.
- Diagonal matrices have eigenvalues equal to the diagonal entries.

Eigenvalues are found from

$$(A - \lambda I)\boldsymbol{v} = 0 \quad \Longleftrightarrow \quad \det(A - \lambda I) = 0.$$

Real eigenvalues

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \Rightarrow (2 - \lambda)(3 - \lambda) = 0.$$

Thus $\lambda_1 = 2$, $\lambda_2 = 3$, with eigenvectors along the coordinate axes.

Rotation and Complex Eigenvalues

$$B = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

This is a 90° rotation.

$$\det(B - \lambda I) = \lambda^2 + 1 = 0 \Rightarrow \lambda = \pm i.$$

There are no real eigenvectors: no real nonzero vector keeps its direction after a 90° rotation.

Multiplicity and Diagonalizability

- Algebraic multiplicity: how many times λ is a root of the characteristic polynomial.
- Geometric multiplicity: dimension of the eigenspace.

$$\text{geometric multiplicity} \leq \text{algebraic multiplicity}.$$

Definition

A matrix is diagonalizable if it has a basis of eigenvectors.

Symmetric matrices are always diagonalizable.

Eigenvalue Properties

- $\text{tr}(A) = \sum_i \lambda_i$.
- $\det(A) = \prod_i \lambda_i$.
- Eigenvectors associated with distinct eigenvalues of a symmetric matrix are orthogonal.
- If A is invertible, eigenvalues of A^{-1} are $1/\lambda_i$.
- If A is triangular, eigenvalues are diagonal entries.

Applications of Eigenvalues

- PCA: eigenvectors of covariance matrix give directions of maximum variance.
- Spectral clustering: eigenvectors of graph Laplacians identify clusters.
- PageRank: dominant eigenvector gives web-page importance.
- Stability analysis: signs/magnitudes of eigenvalues determine convergence.

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

What Is a Matrix Decomposition?

A decomposition writes a matrix as a product of simpler matrices.

Like $12 = 3 \times 4$, a matrix can be factored into structures that are easier to compute with.

Useful for solving systems, least squares, dimensionality reduction, compression, noise removal and large data analysis.

Eigen-Decomposition

If $A \in \mathbb{R}^{n \times n}$ has n linearly independent eigenvectors, then

$$A = PDP^{-1},$$

where

$$P = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n], \qquad D = \text{diag}(\lambda_1, \ldots, \lambda_n).$$

It means:

1. change to eigenvector coordinates using P^{-1};
2. scale by D;
3. return using P.

Powers and Limitations

Eigen-decomposition simplifies powers:

$$A^k = PD^kP^{-1}, \qquad D^k = \mathrm{diag}(\lambda_1^k, \ldots, \lambda_n^k).$$

Limitations:

- defined only for square matrices;
- not every square matrix is diagonalizable;
- many data matrices are rectangular.

SVD avoids these limitations.

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Why SVD?

In ML, data matrices are often rectangular:

$$A \in \mathbb{R}^{m \times n},$$

where m is the number of observations and n is the number of features. SVD applies to every real matrix:

- square or rectangular;
- invertible or singular;
- full rank or rank deficient.

Theorem (Singular Value Decomposition)

For any $A \in \mathbb{R}^{m \times n}$, there exist orthogonal matrices $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$, and a rectangular diagonal matrix $\Sigma \in \mathbb{R}^{m \times n}$ such that

$$A = U \Sigma V^T.$$

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

are the nonzero singular values.

Full and Reduced SVD

Full SVD dimensions:

$$\underbrace{A}_{m\times n} = \underbrace{U}_{m\times m}\,\underbrace{\Sigma}_{m\times n}\,\underbrace{V^T}_{n\times n}.$$

If $\operatorname{rank}(A) = r$, the reduced SVD is

$$A = U_r \Sigma_r V_r^T,$$

where

$$U_r \in \mathbb{R}^{m\times r}, \quad \Sigma_r \in \mathbb{R}^{r\times r}, \quad V_r \in \mathbb{R}^{n\times r}.$$

Meaning of U, Σ, V

$$U = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m], \qquad V = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n].$$

- $\boldsymbol{u}_i$: left singular vectors.
- $\boldsymbol{v}_i$: right singular vectors.
- σ_i: singular values.

They satisfy

$$A\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i, \qquad A^T \boldsymbol{u}_i = \sigma_i \boldsymbol{v}_i.$$

Geometric Interpretation

$$A = U\Sigma V^T.$$

A vector is transformed from right to left:

1. V^T: rotate or reflect the input space;
2. Σ: stretch or contract along orthogonal axes;
3. U: rotate or reflect into the output space.

In 2D, the unit circle is transformed into an ellipse. Singular values give the axes lengths.

Relationship with Eigenvalues

From $A = U\Sigma V^T$:

$$A^T A = V\Sigma^T \Sigma V^T.$$

Therefore:

- columns of V are eigenvectors of $A^T A$;
- eigenvalues of $A^T A$ are σ_i^2;
- $\sigma_i = \sqrt{\lambda_i(A^T A)}$.

Similarly, columns of U are eigenvectors of AA^T.

Steps for Computing SVD

To compute $A = U\Sigma V^T$:

1. Compute $A^T A$.
2. Find eigenvalues λ_i and eigenvectors $\boldsymbol{v}_i$.
3. Set $\sigma_i = \sqrt{\lambda_i}$.
4. Form $V = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n]$.
5. Compute $\boldsymbol{u}_i = \frac{1}{\sigma_i} A\boldsymbol{v}_i$.
6. Build Σ.

Short Calculation Example

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}.$$

Then

$$A^T A = \begin{bmatrix} 9 & 0 \\ 0 & 1 \end{bmatrix}.$$

Eigenvalues are 9 and 1, hence singular values are 3 and 1. Here

$$A = I \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} I^T.$$

SVD as Rank-One Sum

If $\text{rank}(A) = r$, then

$$A = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

Each term $\sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$ is rank one.

Keeping only the first terms gives a compressed approximation of the data.

Rank and Conditioning

- rank(A) is the number of nonzero singular values.
- The condition number is
$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$
- Large $\kappa_2(A)$ means small input errors can create large output errors.

SVD exposes rank deficiency and ill-conditioning clearly.

Moore-Penrose Pseudoinverse

If $A = U\Sigma V^T$, then

$$A^+ = V\Sigma^+ U^T,$$

where Σ^+ replaces every nonzero σ_i by $1/\sigma_i$ and transposes the rectangular shape.

The pseudoinverse gives least-squares and minimum-norm solutions for rectangular or rank-deficient systems.

The best rank-k approximation of A is

$$A_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

Theorem (Eckart-Young-Mirsky)

A_k is the closest rank-k matrix to A in the spectral and Frobenius norms.

Applications: compression, denoising, latent-factor models and PCA.

Choosing the Number of Components

The retained energy is

$$\text{Energy}(k) = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}.$$

Choose the smallest k such that the energy is high, for example 90%, 95% or 99%.

A scree plot helps identify where singular values begin to decrease slowly.

Importance of SVD in ML

- Dimensionality reduction and PCA.
- Data compression and noise reduction.
- Stable least-squares computation.
- Rank detection and multicollinearity diagnosis.
- Recommendation systems through latent factors.

SVD in Python

```python
import numpy as np

X = np.random.randn(100, 40)
X = X - X.mean(axis=0) # center columns

U, s, Vt = np.linalg.svd(X, full_matrices=False)

k = 5
Xk = (U[:, :k] * s[:k]) @ Vt[:k, :]
err = np.linalg.norm(X - Xk, 'fro') / np.linalg.norm(X, 'fro')

print("Singular values:", s[:10])
print("Relative Frobenius error:", err)
```

Least Squares with SVD/Pseudoinverse

```python
import numpy as np

X = np.random.randn(50, 5)
y = np.random.randn(50)

beta_lstsq, residuals, rank, s = np.linalg.lstsq(X, y, rcond=None)
beta_pinv = np.linalg.pinv(X) @ y

print("rank(X) =", rank)
print("singular values =", s)
print("same solution?", np.allclose(beta_lstsq, beta_pinv))
```

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Motivation

High-dimensional data are difficult to visualize, expensive to process and often redundant.

Definition

PCA is a linear transformation technique that finds new orthogonal axes along which the data vary the most, then projects the data onto the leading axes.

PCA preserves as much variance as possible while reducing dimension.

Geometric and Statistical Intuition

- PC1: direction of maximum variance.
- PC2: orthogonal direction with next-largest variance.
- Higher PCs: progressively smaller variance.

PCA finds a projection

$$Z = XW_k,$$

where W_k contains the first k principal directions.

PCA Algorithm

Given centered data $X \in \mathbb{R}^{n \times d}$:

1. Compute covariance matrix: $C = \frac{1}{n-1} X^T X$.
2. Solve $C\boldsymbol{v} = \lambda \boldsymbol{v}$.
3. Sort eigenvectors by decreasing eigenvalues.
4. Keep $W_k = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k]$.
5. Project data: $Z = XW_k$.

Explained Variance

Let $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ be eigenvalues of the covariance matrix. The explained variance ratio of the first k components is

$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{d} \lambda_i}.$$

- Eigenvalues measure variance along PCs.
- Choose k so that cumulative variance is sufficiently high.

Worked Numerical Example

Data points:

$$(2,0), \qquad (0,2), \qquad (3,3).$$

Mean:

$$\bar{\boldsymbol{x}} = (1.67, 1.67).$$

Centered points:

$$(0.33, -1.67), \quad (-1.67, 0.33), \quad (1.33, 1.33).$$

Covariance matrix:

$$C = \begin{bmatrix} 2.33 & 1.17 \\ 1.17 & 2.33 \end{bmatrix}.$$

Worked Example: Interpretation

For

$$C = \begin{bmatrix} 2.33 & 1.17 \\ 1.17 & 2.33 \end{bmatrix},$$

we obtain

$$(\lambda - 3.5)(\lambda - 1.17) = 0.$$

Thus

$$\lambda_1 = 3.5, \qquad \lambda_2 = 1.17.$$

The first principal direction is along $(1, 1)$, and it explains approximately

$$\frac{3.5}{3.5 + 1.17} \approx 75\%$$

of the total variance.

PCA via SVD

For centered data

$$X = U\Sigma V^T.$$

Then:

- columns of V are principal directions;
- variances are $\lambda_i = \frac{\sigma_i^2}{n-1}$;
- projection onto the first k columns of V gives PCA coordinates.

SVD-based PCA is more stable than directly diagonalizing $X^T X$.

PCA in Python

```python
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = np.random.randn(500, 50)
Xz = StandardScaler(with_mean=True, with_std=True).fit_transform(X)

pca = PCA(n_components=0.95, svd_solver='full')
Z = pca.fit_transform(Xz)

print("Selected components:", pca.n_components_)
print("Explained variance:", pca.explained_variance_ratio_)
print("Cumulative:", np.cumsum(pca.explained_variance_ratio_))
```

PCA Applications

- Visualization: reduce many features to 2D or 3D.
- Noise reduction: discard low-variance components.
- Preprocessing: reduce multicollinearity before regression or clustering.
- Face recognition: eigenfaces.
- Recommender systems: latent factors.

Exercise: Blood Pressure PCA

$$X = \begin{bmatrix} 126 & 78 \\ 128 & 80 \\ 128 & 82 \\ 130 & 82 \\ 130 & 84 \\ 132 & 86 \end{bmatrix}.$$

Tasks:

1. Center the data.
2. Compute $C = \frac{1}{n-1} X_c^T X_c$.
3. Find eigenvalues and eigenvectors.
4. Interpret the largest eigenvalue and its eigenvector.

Exercise Interpretation Guide

- Which eigenvalue is largest?
- What percentage of total variance does it explain?
- What does the associated eigenvector represent for systolic and diastolic pressure?
- How is this related to correlation between the two variables?

If both pressures increase together, PC1 often represents global blood-pressure level.

Outline

1. Introduction
2. Vectors, Matrices and Tensors
3. Matrix Arithmetic
4. Linear Transformations
5. Eigenvalues and Eigenvectors
6. Matrix Decompositions
7. Singular Value Decomposition (SVD)
8. Principal Component Analysis (PCA)
9. Conclusion

Mathematical Takeaways

- Vectors, matrices and tensors represent data.
- Matrix arithmetic describes operations and transformations.
- Determinant, rank and inverse describe invertibility and information loss.
- Eigenvalues/eigenvectors reveal invariant directions.
- SVD gives universal, stable matrix factorization.
- PCA applies these concepts to dimensionality reduction.

Machine Learning Takeaways

- Linear algebra is the language of high-dimensional data.
- Neural networks depend on matrix multiplication.
- Regression depends on solving linear systems and least squares.
- SVD and PCA are essential for compression, denoising and visualization.
- Numerical stability matters: avoid unnecessary explicit inverses.

Thank You!

Questions?

